Campus Lost & Found Intelligent Matching System

Final project report

DBMS Experiential Learning Level 3 | 2 October 2026

Member	SRN
Harshit Chandak	PES2UG24CS185
Hamza Shabir Sahapurwala	PES2UG24CS177

A connected recovery workflow using MySQL for relational decisions and MongoDB for flexible details and explainable matching. This report records delivered behavior, genuine verification and remaining human submission actions.

GitHub repository

Submission status: ready after genuine member Git contribution and course-specific manual confirmations.

Report contents

1 Abstract
2 Problem statement
3 Objectives
4 Scope
5 Functional requirements
6 Non-functional requirements
7 Security objectives
8 Technology stack
9 Database justification
10 System architecture
11 ER design
12 Final relational schema
13 Data dictionary
14 MongoDB model
15 SQL implementation and DDL
16 DML and queries
17 Advanced concepts
18 Transactions
19 Views
20 Stored procedures
21 Stored function
22 Triggers
23 Indexes and search
24 MongoDB aggregation
25 Intelligent matching
26 Authentication and authorization
27 Validation and security
28 Outbox consistency
29 Screenshots
30 Testing strategy
31 Test results
32 Results and discussion
33 Team contributions
34 Limitations
35 Future enhancements
36 Conclusion
37 References

1 Abstract

Campus belongings are often reported through disconnected messages. This system connects lost and found reports, ranks plausible pairs using relational facts and document attributes, and requires staff review plus ownership evidence before returning an item. MySQL stores identity, relationships and decisions; MongoDB stores flexible enrichment, comparison explanations and activity. Next.js supplies Student, Staff and Admin interfaces.

The implementation includes ten domain tables, two internal SQL tables, four MongoDB collections, two views, two transactional procedures, one stored function and eight triggers. Verification passed 28 unit tests, 46 live integration checks, a production build and fresh/repeated database setup. Individual Git contributions and course submission formalities remain team actions. No trained model, vectors or measured matching accuracy is claimed.

2 Problem statement

A campus needs one place to report missing belongings and returned items. Similar names, incomplete descriptions and repeated messages make comparison difficult. Returning an item solely because its public description resembles a report can expose private identifiers and cause an incorrect handover. The application must connect potential reports, explain their resemblance, restrict private information and record an authorized ownership decision.

3 Objectives

Provide structured reporting, searchable found items and private lost-report tracking. Combine SQL category/location/time with MongoDB tags and keywords to suggest inspectable matches. Enforce one accepted pair per report and one approved claim per found item. Commit recovery decisions and student notifications together. Document required DBMS concepts with executable examples and captured results.

4 Scope

The system covers campus identity/reference data, lost/found CRUD, weighted candidates, manual pairing, review, owner-only claims, collection closure, notifications and dashboards. Google identity registration and an explicitly enabled local demo login are supported. Media fields store references. Binary uploads, object recognition, email delivery, deployment and semantic vectors are future work.

5 Functional requirements

ID	Behavior	Implementation
FR1	Registered sign-in and current role permissions	Session/OAuth routes; SQL role refresh
FR2	Maintain permitted profile fields	Profile API; no self-edited role
FR3	Report, retrieve, filter and maintain belongings	Lost/found pages and report service
FR4	Generate/explain candidate pairs	Matching service and match_evidence
FR5	Staff accept/reject current candidates	sp_review_match and review page
FR6	Accepted lost owner submits private proof	Claim service and form
FR7	Staff process ownership and notify owner	sp_process_claim and notifications
FR8	Observe recovery and retry enrichment	SQL views, Mongo aggregation and outbox
FR9	Admin manages references/accounts	Admin management interface

Edits/deletes are guarded by ownership and state. Supported interfaces preserve terminal decisions.

6 Non-functional requirements

Requirement	Implemented measure and evidence limit
Consistency	Transactions, row locks, unique success guards; concurrency tests
Recoverability	Durable outbox retains failed payloads; replay test
Privacy	Ownership scopes and private found/claim fields
Maintainability	Typed services, validated inputs, pinned installation
Reproducibility	Fresh/repeated setup and seeds verified
Usability	Forms, tables, filters and three dashboards; actual captures
Performance control	Lists capped at 500; matching at 2,000 eligible comparisons
Time clarity	Offset-aware input, UTC storage and Asia/Kolkata display

No load benchmark, uptime SLA or user study is claimed. Caps are safeguards rather than performance measurements.

7 Security objectives

Authenticate requests; refresh role/account existence from MySQL; restrict decisions to Staff/Admin; enforce owner-only claims; reject cross-origin mutations; parameterize SQL; validate at API/database layers; and exclude real environment credentials from Git and packaging. Application passwords are not stored: Google authenticates registered identities, while demonstration sessions require explicit loopback opt-in.

Machine credentials remain in an ignored local environment file. An external installation requires HTTPS, appropriate database accounts and disabled demo login.

8 Technology stack

Next.js 15.5.26 App Router/Route Handlers, React 19.3.0, TypeScript 5.9.3, MySQL 8 through mysql2 3.24.4, MongoDB through driver 6.21.0, Zod 3.25.76, jose 5.10.0 and Vitest 5.0.3. package-lock.json pins dependencies. Node 22.12+ is required; verification used Node 24 and MySQL 8.0.46.

9 Database justification

MySQL fits reference entities, user relationships, report lifecycle and recovery decisions. Foreign keys preserve connected records; transactions close both reports and create notifications as one decision.

MongoDB fits category-dependent scalar attributes, tags, keywords, media references and detailed scoring evidence. A wallet can record material while headphones record model information without requiring a SQL column for each category. MySQL remains authoritative for ownership/status. The faculty SQL + NoSQL option is satisfied. Vector search is recommended but optional and is not implemented.

10 System architecture

The browser submits to Next.js route handlers. Session checks identify the actor; Zod and services validate input, ownership and references. Services use parameterized SQL and authorized document reads. SQL changes enqueue MongoDB payloads; an ordered worker applies them. Dashboards combine SQL views and MongoDB aggregation.

Browser -> Next.js pages/API -> authenticated services -> MySQL records/outbox -> replay worker -> MongoDB documents.

Architecture.md (implementation/Architecture.md) contains flow/state diagrams. API_Endpoints.md (implementation/API_Endpoints.md) lists routes and permissions.

11 ER design

The approved ER image (design/ER_Diagram.png) and editable diagram (design/ER_Diagram.drawio) remain unchanged. CAMPUS groups DEPARTMENT and LOCATION; USER reports items and receives NOTIFICATION; CATEGORY/LOCATION classify reports; MATCH relates LOST_ITEM and FOUND_ITEM; CLAIM records submissions for a found item.

Reports can have several candidates but one accepted pair. A found item can have several claims but one approval. Nullable reviewer/processor fields represent pending decisions. Final implementation additions

are documented separately: internal queue/migration tables, revisions and generated unique fields. ThePDF appendix displays the original ER plus enlarged extracts.

12 Final relational schema

The canonical schema is generated from a fresh database installed using the submitted setup script and all migrations.

Domain table	Purpose
CAMPUS	Campus identity/address
DEPARTMENT	Campus department
USER	Registered identity, department and role
CATEGORY	Report classification
LOCATION	Campus location
LOST_ITEM	Missing-item facts, state and revision
FOUND_ITEM	Finder's facts, state and revision
MATCH	Pair, score, method, decision, actor and version snapshots
CLAIM	Ownership evidence and processing decision
NOTIFICATION	Recipient message and read state

MONGO_OUTBOX stores ordered payloads, attempts, errors and processing times. SCHEMA_MIGRATION stores applied migration names, checksums and times. Both are internal infrastructure rather than new campus entities.

LOST_ITEM/FOUND_ITEM revision starts at 1 and application edits increment it. MATCH lost_revision/found_revision snapshot compared versions. Generated accepted_lost_id/accepted_found_id contain linked IDs only for ACCEPTED rows and have separate unique keys. CLAIM approved_found_id contains found_id only for APPROVED rows and is unique. These guards prevent overlapping successful decisions.

The Markdown schema (design/Relational_Schema.md), Word schema (design/Relational_Schema.docx), schema PDF (design/Relational_Schema.pdf) and fresh metadata (testing/results/final-schema.json) list all columns/types/defaults/nullability, PK/FK/unique keys, CHECKs, generated expressions and indexes. Domain FK updates/deletes use RESTRICT. Fresh identities are unsigned AUTO_INCREMENT; compatibility retains legacy signed key types. Windows metadata stores equivalent lowercase table names.

13 Data dictionary

Data_Dictionary.md (design/Data_Dictionary.md) lists exact fields/constraints for all twelve tables and relationship cardinalities. USER.department_id and pending reviewer/processor fields are optional. Report owner/category/location references are required. MongoDB linked IDs are service-validated rather than cross-database foreign keys. SQL BOOLEAN is physically tinyint(1). Dates persist as UTC and display in IST.

14 MongoDB model

Collection	Linked ID and content	Indexes
lost_item_details	lost_id; tags, keywords, scalar attributes, media, timestamps	Unique ID, tags multikey, text
found_item_details	found_id; enrichment and private identifiers	Unique ID, tags multikey, text
match_evidence	match_id; score, rules, tags, snapshots, explanation/version	Unique ID, descending score
audit_events	Actor/entity IDs, event, payload and time	User/time, event/time, entity

model.mjs is authoritative; setup.js is its exported shell form. Setup creates collections or updates validators with collMod and installs indexes. Validation is strict: tags are strings, timestamps BSON dates, and scores constrained. Linked IDs accept positive BSON numeric types; SQL/service checks establish valid identities. Found private details are restricted to reporter and Staff/Admin.

Media are URL references, not uploaded binaries. Actual validators, indexes and sample documents appear in database evidence (testing/results/database-evidence.json); MongoDB_Model.md (implementation/MongoDB_Model.md) explains retrieval.

15 SQL implementation and DDL

schema.sql creates domain tables. indexes_views.sql, advanced.sql, revisions.sql and procedures.sql install reporting, outbox, integrity guards, revisions and decision routines. Setup runs them in order and records migrations.

ddl_demo.sql executes CREATE, ALTER and DROP on an isolated demonstration table; evidence records output without deleting domain records. MySQL DDL can implicitly commit, so setup is not one rollback-safe transaction. Migration names prevent repeats. Checksums are stored but not compared with previously applied content.

16 DML and queries

Report services implement INSERT, SELECT, guarded UPDATE and DELETE. Decisions update connected records and insert notifications. Live tests verify persistence in both stores and deletion of unmatched OPEN reports. Mongo examples use a uniquely named scratch collection.

Query	Concepts
Q1	Lost owner/category/location JOIN and ORDER BY
Q2	Status GROUP BY and COUNT
Q3	Reporter GROUP BY with HAVING active count >= 2
Q4	Aggregate subquery; above-average reporting
Q5	Unread notification grouping
Q6	User-summary view
Q7	Candidate JOIN, ranking and function
Q8	COUNT/AVG/MAX/MIN/SUM and HAVING
Q9	Claimant and processor JOIN
Q10	NOT EXISTS for reports without pending candidates
Q11	Recovery view and percentage
Q12	Pending outbox grouping/retry totals

All twelve examples in queries.sql (../database/mysql/queries.sql) executed successfully. An empty result can be valid for the small dataset and is not represented as a populated output.

17 Advanced concepts

The project demonstrates more than four required advanced categories: transactions, views, stored procedures/functions, triggers, indexing/full-text retrieval and NoSQL aggregation. Each supports reporting, recovery integrity or document retrieval. The following sections identify actual installed behavior.

18 Transactions

sp_review_match locks LOST_ITEM, FOUND_ITEM and MATCH in that order, checks role/OPEN state/current revisions, accepts a pair, marks reports MATCHED, rejects competitors and inserts its notification before COMMIT. Exceptions ROLLBACK.

sp_process_claim verifies the accepted owner, prevents self-processing, locks related records, approves collection, closes both reports, rejects competing claims and inserts collection instructions atomically. Concurrency tests produced one accepted pair and one committed approval; stale versions were rejected without partial changes.

transactions.sql (../database/mysql/transactions.sql) separately executes START TRANSACTION, SAVEPOINT, ROLLBACK TO SAVEPOINT and ROLLBACK with disposable examples. SQL atomicity does not extend to MongoDB.

19 Views

vw_user_lost_summary centralizes per-user totals/states for Student metrics and Q6. vw_recovery_dashboard aggregates category-level report/recovered counts for Staff analytics and Q11. Both installed views were queried and their output captured.

20 Stored procedures

sp_review_match and sp_process_claim encapsulate authorized multi-table decisions. Role, state, revision and ownership checks combine with locks, unique guards and exception handlers. Services call these routines instead of issuing independent decision writes.

21 Stored function

fn_match_band returns HIGH for score >= 0.80, POSSIBLE for >= 0.55, otherwise LOW. Q7 calls it for candidate display. The band explains a heuristic score; it is not a measured ownership probability.

22 Triggers

Eight insert/update triggers reinforce direct database input: four require supplied MATCH reviewers/CLAIM processors to be STAFF or ADMIN; four reject report event times over one minute in the future. Integration checks rejected direct invalid role/date writes. Revisions are application-maintained, not trigger-incremented.

23 Indexes and search

SQL indexes support owner/status/time/category/location lists, unread notifications, ranking and claims. Foreign keys and unique keys index relationships and pairs. Generated uniqueness permits multiple NULL pending rows while preventing overlapping success.

MongoDB uses unique linked IDs, tags multikey, tags/keywords text, score and audit indexes. demo_queries.js demonstrates text/tag retrieval. No latency improvement or index benchmark is claimed.

24 MongoDB aggregation

Dashboard/demo pipelines group audit activity, compute mean evidence scores and unwind/group matched tags. Captured output demonstrates $match, $group, $unwind, $sort and aggregation operators. SQL handles relational recovery totals; MongoDB handles document evidence/activity.

25 Intelligent matching

Eligibility requires OPEN reports in the same campus/category, with found time between five minutes before lost time and fourteen days after it. Unicode-normalized token sets and exact SQL facts contribute weighted similarity.

Rule	Weight
Category	25%

Rule	Weight
Name	20%
Description	10%
Color	10%
Brand	10%
Location	10%
Time proximity	5%
MongoDB tags/keywords	10%

Automatic candidates require >= 0.55. Missing values score zero; repeated tokens do not inflate similarity. Manual pairs still require review/state/version checks. Generation is idempotent and preserves terminal decisions. Evidence stores rule contributions, non-private snapshots, matched tags and version.

Intelligent means explainable ranking and workflow support. No trained AI or embeddings are used; precision/recall was not measured without a labeled campus dataset. A score suggests a candidate, not ownership proof.

26 Authentication and authorization

Google OAuth validates short-lived state and verified registered email. The app issues an eight-hour HS256 session in an HttpOnly, SameSite=Lax cookie and refreshes account/role from SQL on every request. Live tests reject tampered tokens and stale admin claims.

STUDENT manages permitted own records and submits proof. STAFF generates/reviews matches and processes eligible claims. ADMIN also registers identities/manages references. Demo login requires explicit enablement and loopback conditions. External Google exchange remains untested with valid credentials.

27 Validation and security

Zod validates required fields, lengths, states, timestamps and detail structures. Services check ownership, references, duplicates and workflow permission. SQL keys/CHECKs/triggers constrain connected data, roles, scores and times. MongoDB rejects malformed documents. RESTRICT preserves dependent history.

Mutation Origin checks, parameterized SQL and field allowlists reduce unsafe input. Private attributes/remarks and claim proof are scoped. Integration covers cross-origin rejection, role isolation, forged sessions, invalid direct DB writes and notification ownership. Administrative raw SQL access can bypass some supported workflow rules and must be restricted operationally.

28 Outbox consistency

A report transaction writes core SQL state and a MongoDB payload to MONGO_OUTBOX together. A named SQL lock permits one ordered worker. Idempotent UPSERT/DELETE operations set processed_at only on success; failures retain payload/error, increment attempts and stop later writes until retry.

This is recoverable eventual consistency, not a distributed transaction. Matching waits for unsynchronized detail writes. Staff retry or npm run db:sync replays the queue. Integration retained, repaired and replayed a failed payload. document_id is polymorphic, not FK, since a delete can outlive its domain row. Audit writes are separately best effort.

29 Screenshots

Fourteen actual local production captures show sign-in, Student metrics, Staff candidates, scoring, claim form/approval, CLOSED found report, notifications, lost-report form, filtering, Admin management, recovery metrics and mobile dashboard. Data are synthetic campus examples. The gallery (screenshots/README.md) identifies each state.

The found-report form was additionally captured and visually checked during cleanup as 14-found-report-form.jpg. Found persistence is covered by live integration and its closed report is also captured. The PDF includes selected actual screenshots.

30 Testing strategy

Unit tests cover eligibility/weights/normalization, payload validation, permissions and workflow boundaries. Live checks use the production app and actual databases, generate uniquely labeled fixtures and clean only their own records. Schema checks create disposable named databases and run setup/reference/demo seeding twice.

Positive checks cover both-store persistence, evidence, owner claim, closure, notifications, views and function. Negative checks cover sessions, role, origin, references/times, duplicates, privacy, stale acceptance and unauthorized decisions. Concurrency and failed-write replay are explicit cases. See Test_Cases.md (testing/Test_Cases.md).

31 Test results

Check	Cleanup result
npm ci	PASS
TypeScript	PASS
Unit tests	PASS, 28 of 28
Production build	PASS
Live integration	PASS, 46, zero failures
Fresh/repeated SQL setup and seeds	PASS, 12 tables, 2 views, 3 routines, 8 triggers
MongoDB setup/validation/retrieval/aggregation	PASS, 4 collections
Dependency audit	PASS, zero registry advisories at check time
External Google login	NOT RUN - external credentials unavailable
Word page rendering	NOT RUN - LibreOffice dependency unavailable

Testing_Results.md (testing/Testing_Results.md) links exact UTC timestamps and machine evidence. Tests demonstrate covered local correctness, not production load, real-world accuracy or complete security assurance.

32 Results and discussion

The tested workflow connects both databases and closes recovery through an authorized SQL transaction. Concurrent decisions preserve successful-pair uniqueness; changed reports block stale acceptance. MongoDB contributes flexible descriptions and explanations; the outbox makes secondary-write failure recoverable.

The revised schema/dictionary include fields absent from the earlier document. Canonical fresh metadata and legacy compatibility notes describe actual installation behavior. Academic readiness still requires member-authored commits and human approval/demo/originality confirmations.

33 Team contributions

Member	Database responsibilities	Application responsibilities
Harshit Chandak, PES2UG24CS185	CAMPUS/DEPARTMENT/USER/LOST_ITEM/NOTIFICATION; summary view; Mongo lost/audit; Q1-Q6	Auth, profile, lost CRUD, Student dashboard/inbox
Hamza Shabir Sahapurwala, PES2UG24CS177	CATEGORY/LOCATION/FOUND_ITEM/MATCH/CLAIM; recovery view; procedures/function/triggers; Mongo found/evidence; Q7-Q12	Found CRUD, scoring/review, claims, Staff/Admin interface

ER/schema/dictionary, architecture, outbox, reference screens, security, setup, integration tests, report and demo are shared. The existing work division (planning/Team_Work_Division.md) remains the source of responsibility.

Read-only Git verification found nine commits, all attributed to Harshit; local HEAD and remote main matched b68b64425da2b385c4d5e2d7eb6883aab6b3b0cd. No Hamza commit was verified. Scope records are not proof of personal authorship of automated work. Both members must review and push genuine assigned work using their own accounts. The handoff (planning/GitHub_Jira_Plan.md) maps files/actions. Jira is optional planning, not a faculty requirement.

34 Limitations

No labeled accuracy evaluation, production deployment, load benchmark or user study. Lists/comparisons are capped; media are references. Cross-store updates are eventual and audit best effort. Migration checksums are stored but not compared; partly applied DDL is not automatically reversed. External Google login and Word page rendering remain unverified.

35 Future enhancements

Add pagination, labeled-data calibration, optional evaluated embeddings, secure upload, email, durable audit, monitoring and migration recovery. Test deployment-specific HTTPS/identity. Future scope is separate from delivered features.

36 Conclusion

The system implements connected campus recovery with justified SQL + NoSQL use. Explainable ranking narrows candidates, staff verification establishes ownership, and transactional procedures keep decisions consistent. Source, updated design, executed queries, tests and screenshots support the demonstration. Genuine Git and course confirmation actions are recorded rather than fabricated.

37 References

• Faculty requirements (requirements/DBMS_Level_3_Requirements_Draft.docx), supplied draft.

• MySQL 8.0 transactions.

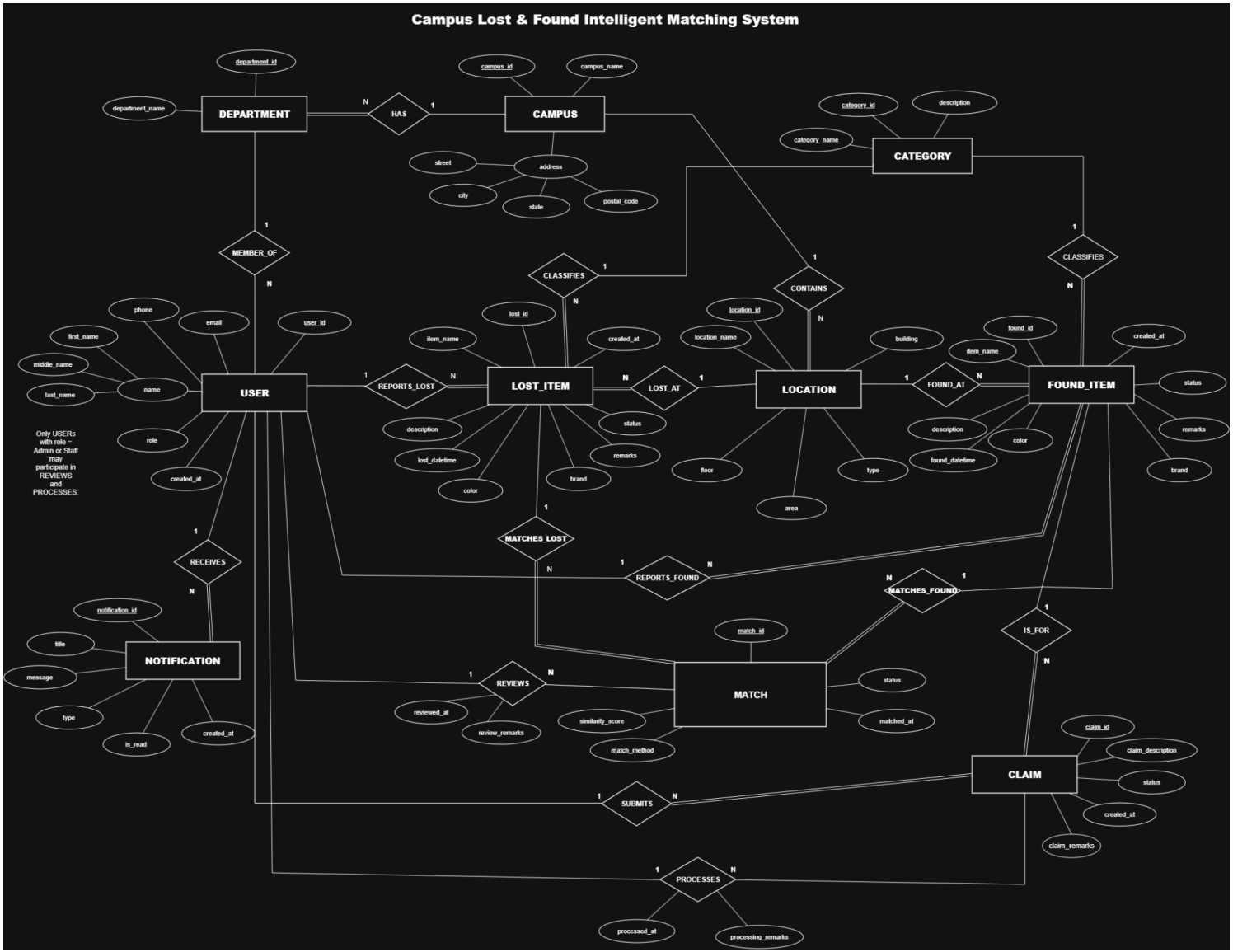
• MongoDB schema validation.

• Next.js Route Handlers.

• Submitted SQL, Mongo model, services/setup/seeds and actual test/schema results are the primary basis for implementation claims.

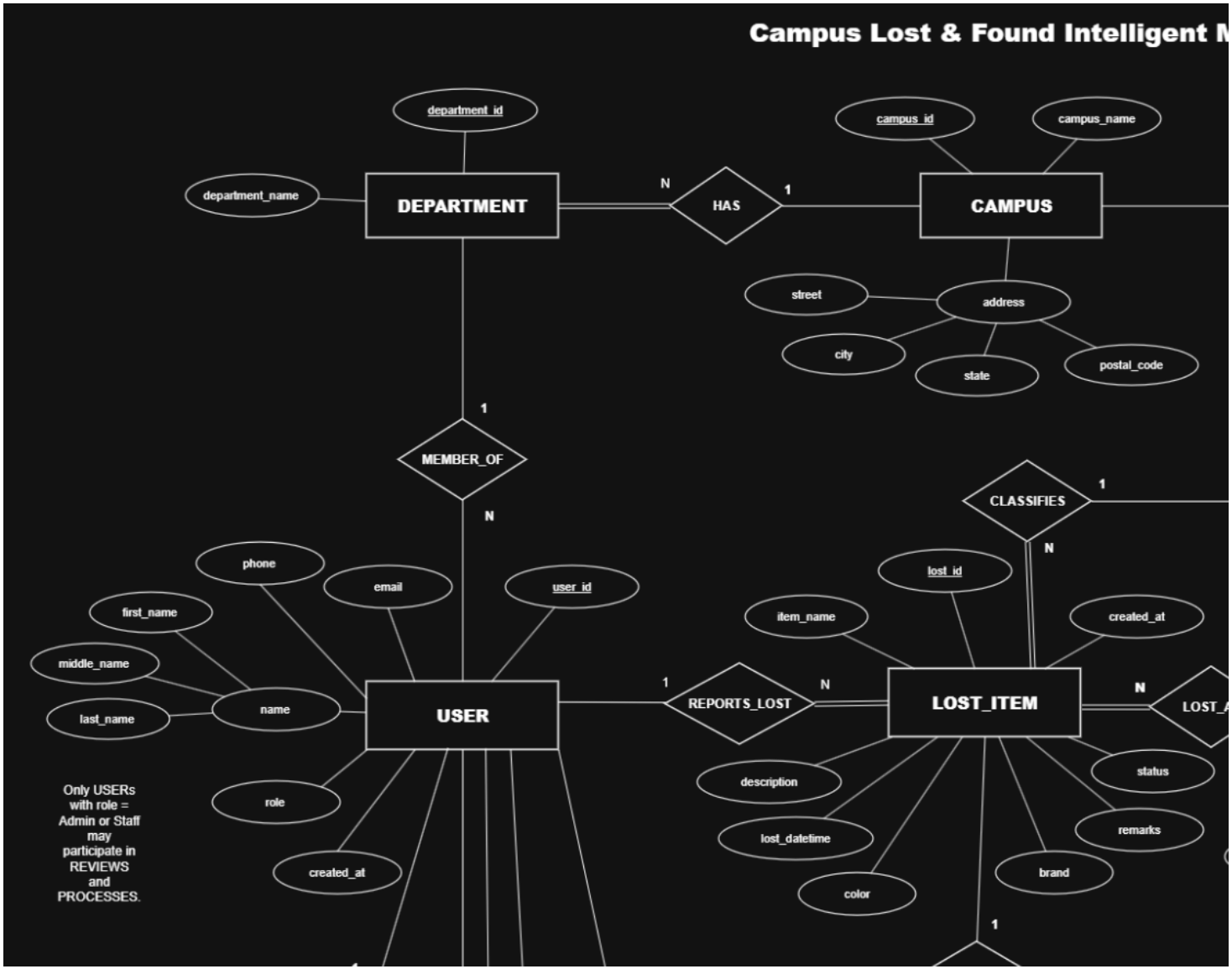
Appendix A Original approved ER diagram

Unchanged conceptual design. Final implementation additions are listed in section 12 and the schema. The next pages enlarge overlapping areas of the same image.



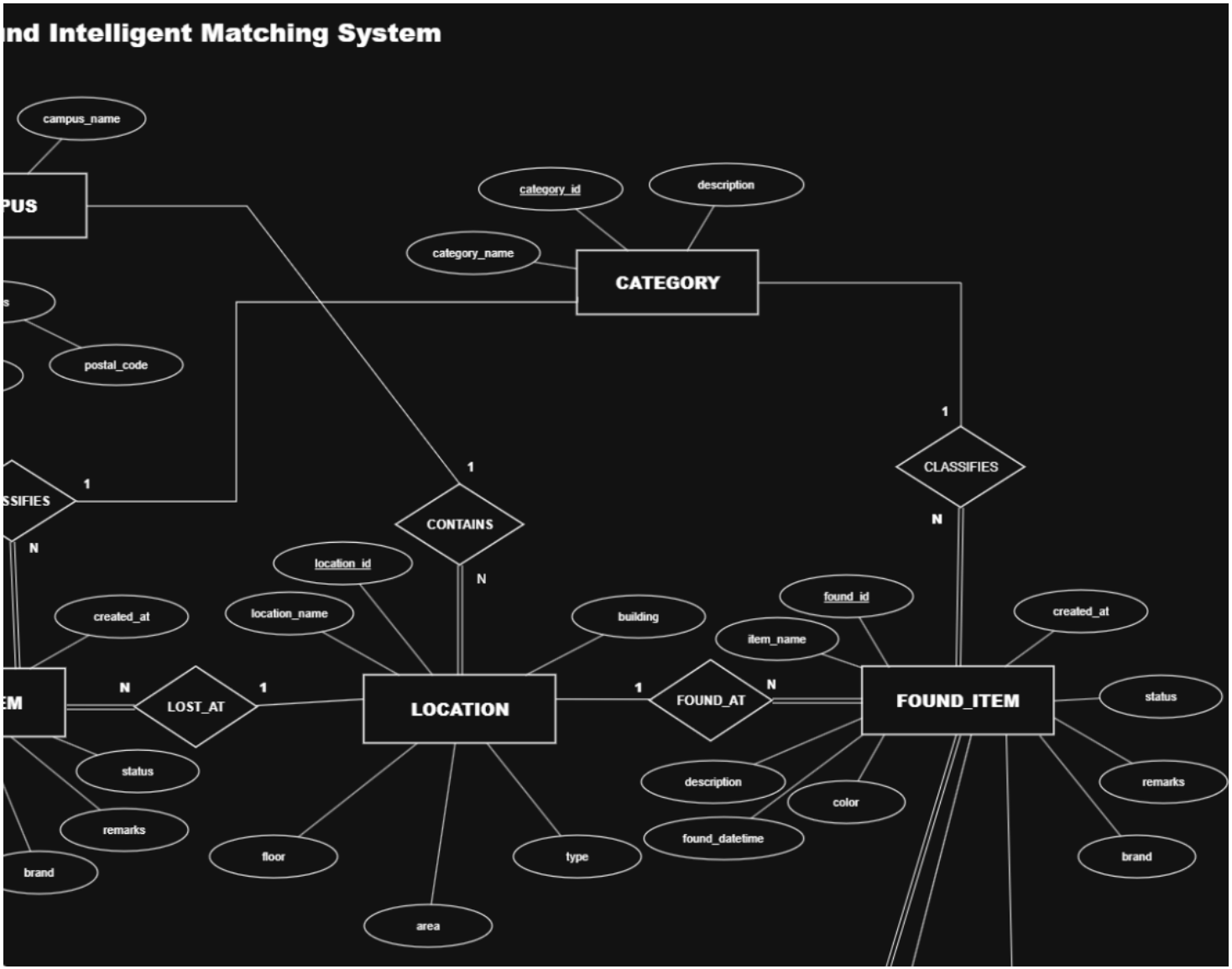
ER detail upper left

Magnified extract of the unchanged ER image. Overlap preserves relationship context.



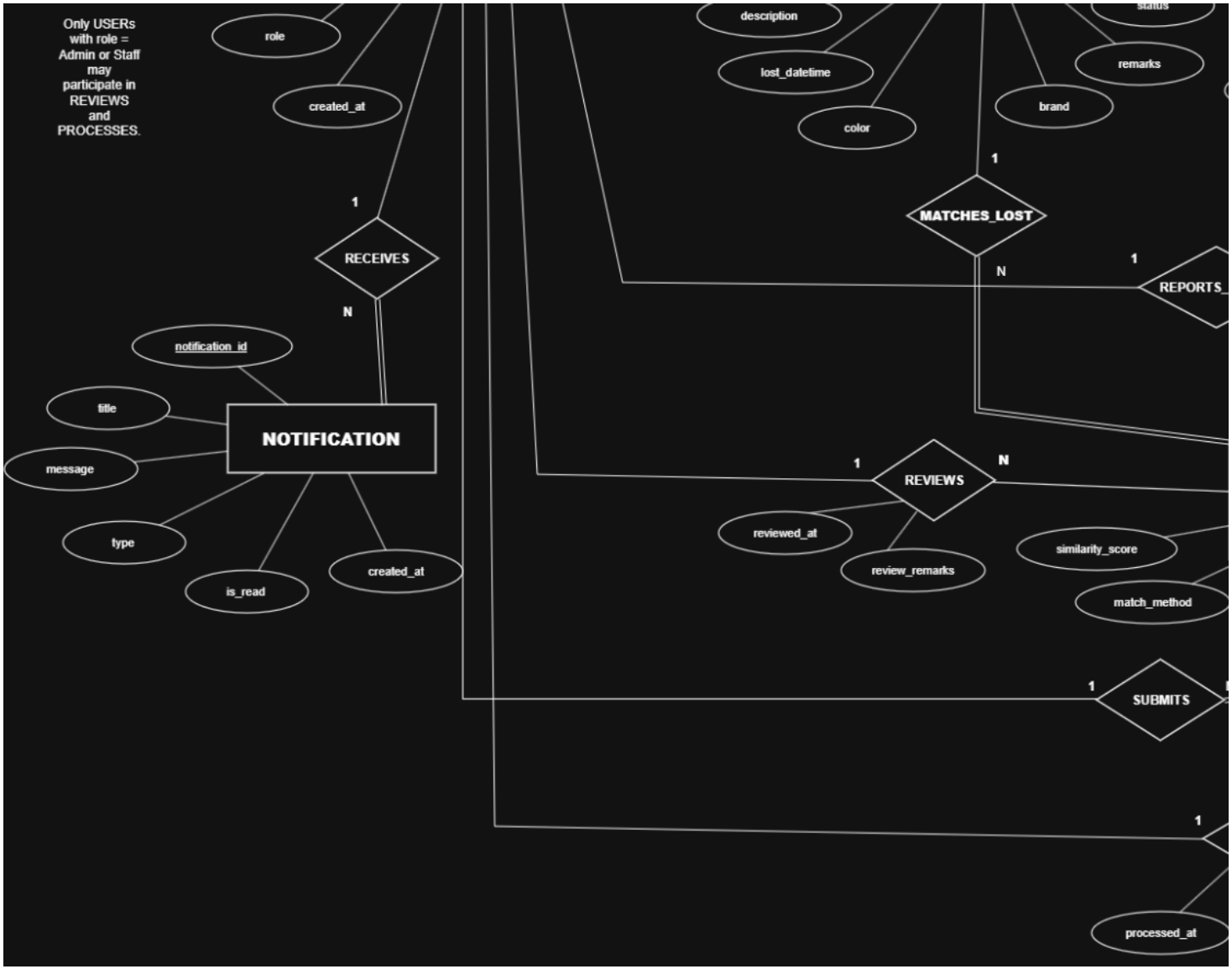
ER detail upper right

Magnified extract of the unchanged ER image. Overlap preserves relationship context.



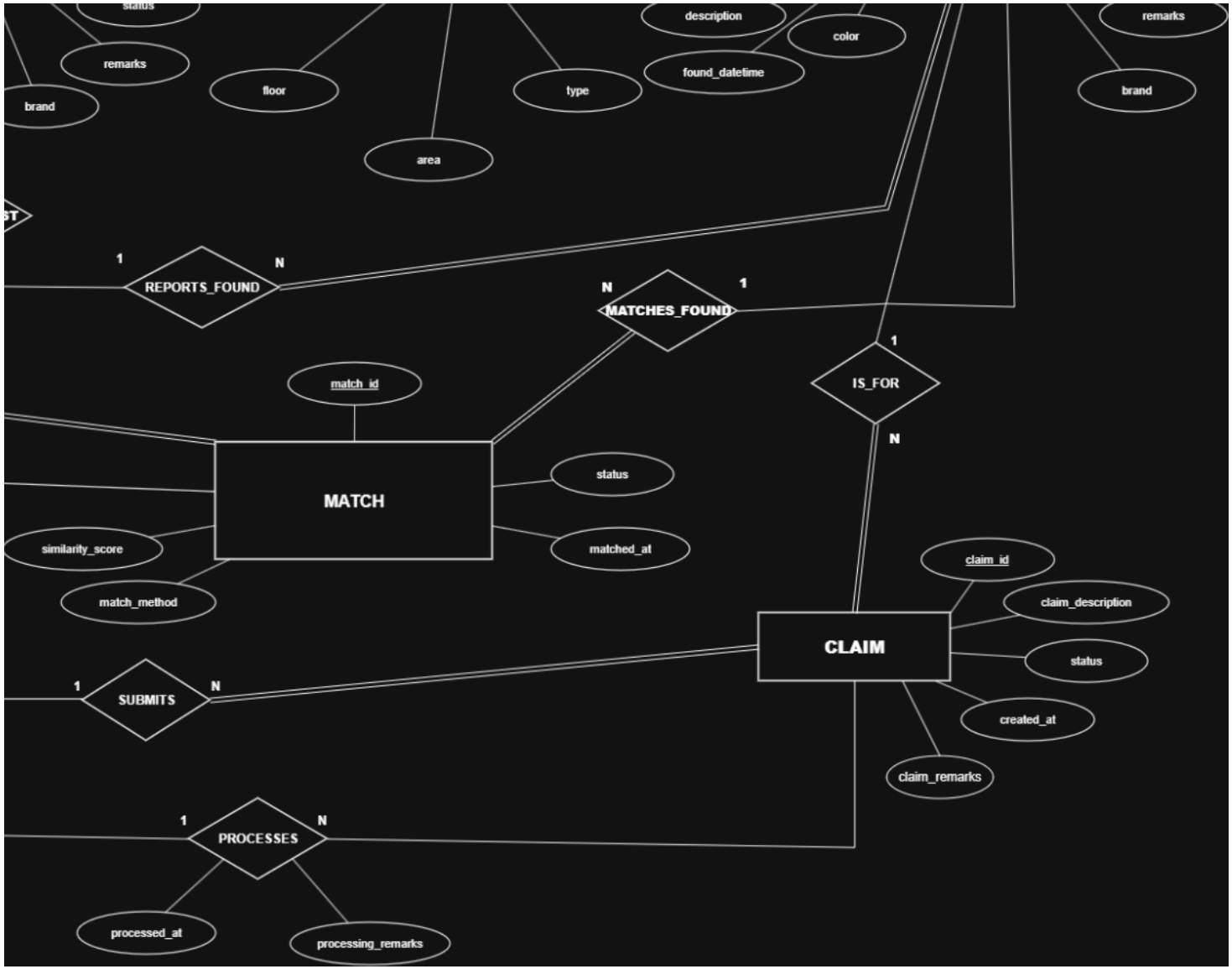
ER detail lower left

Magnified extract of the unchanged ER image. Overlap preserves relationship context.



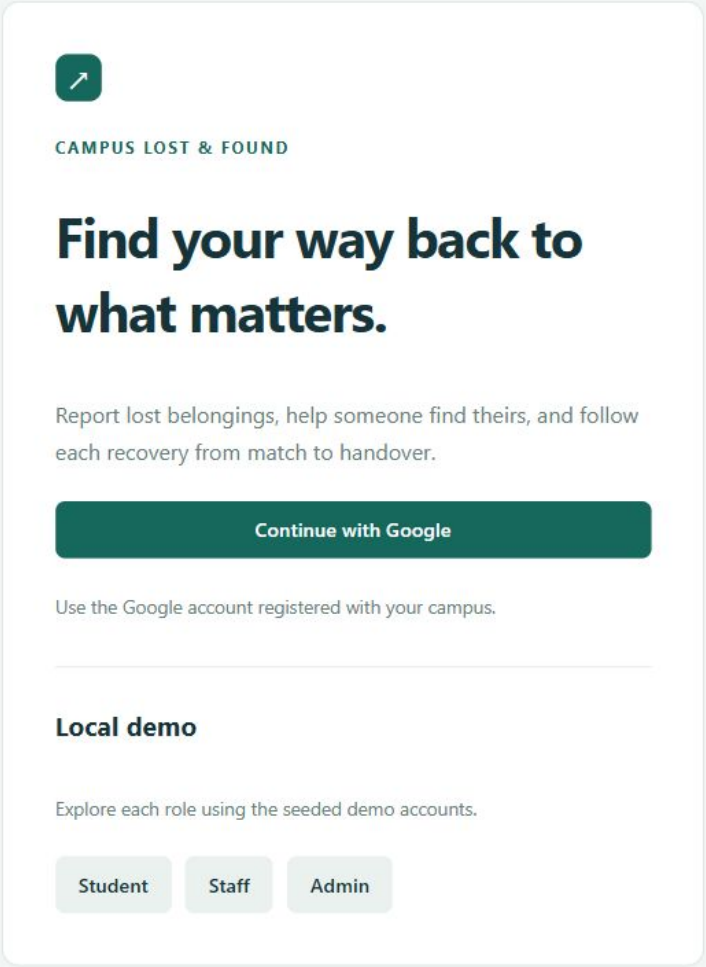
ER detail lower right

Magnified extract of the unchanged ER image. Overlap preserves relationship context.



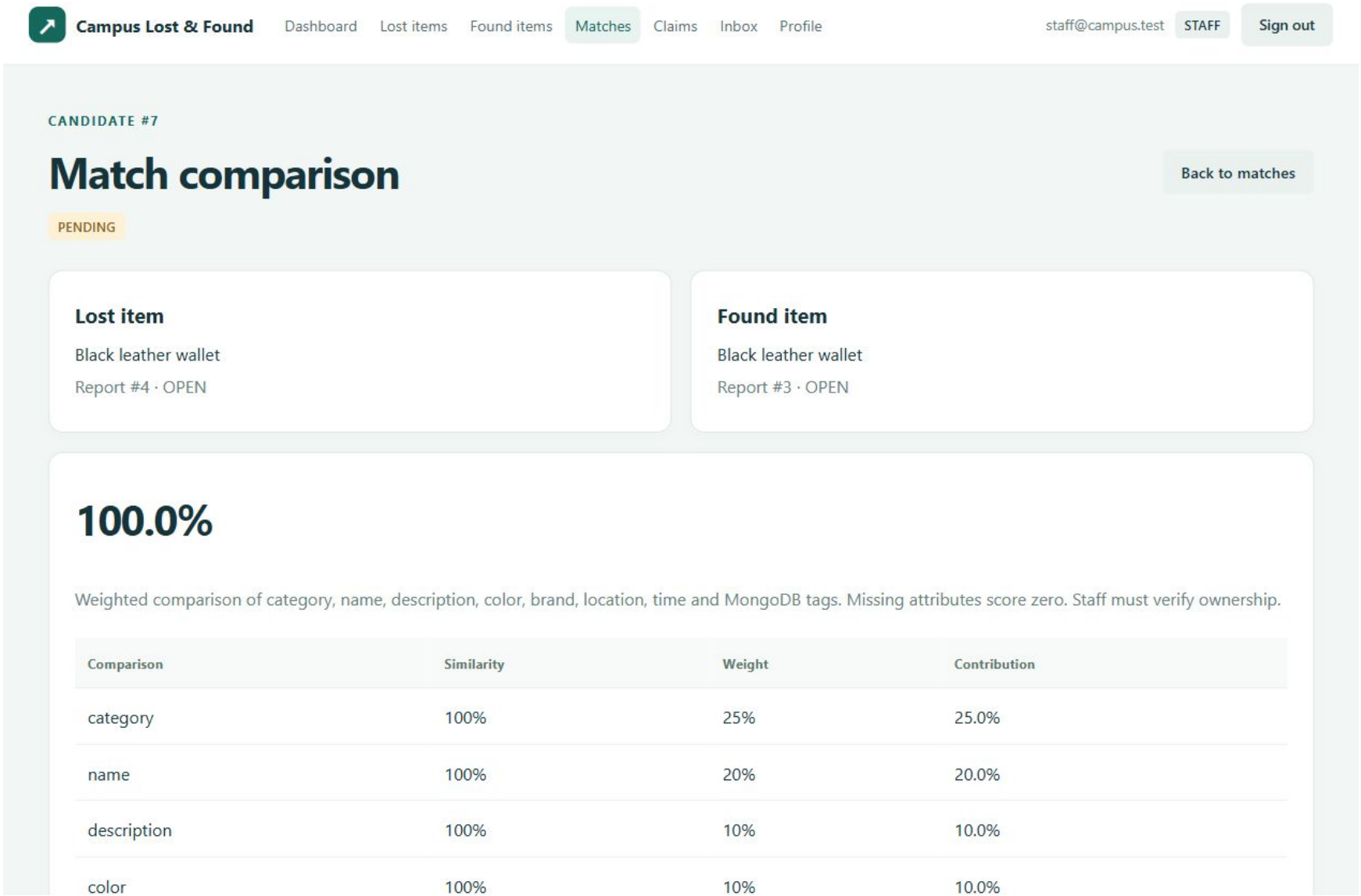
Appendix B Role sign in

Actual browser evidence: 01-sign-in.jpg. Continuous excerpts; original file included in docs/screenshots.



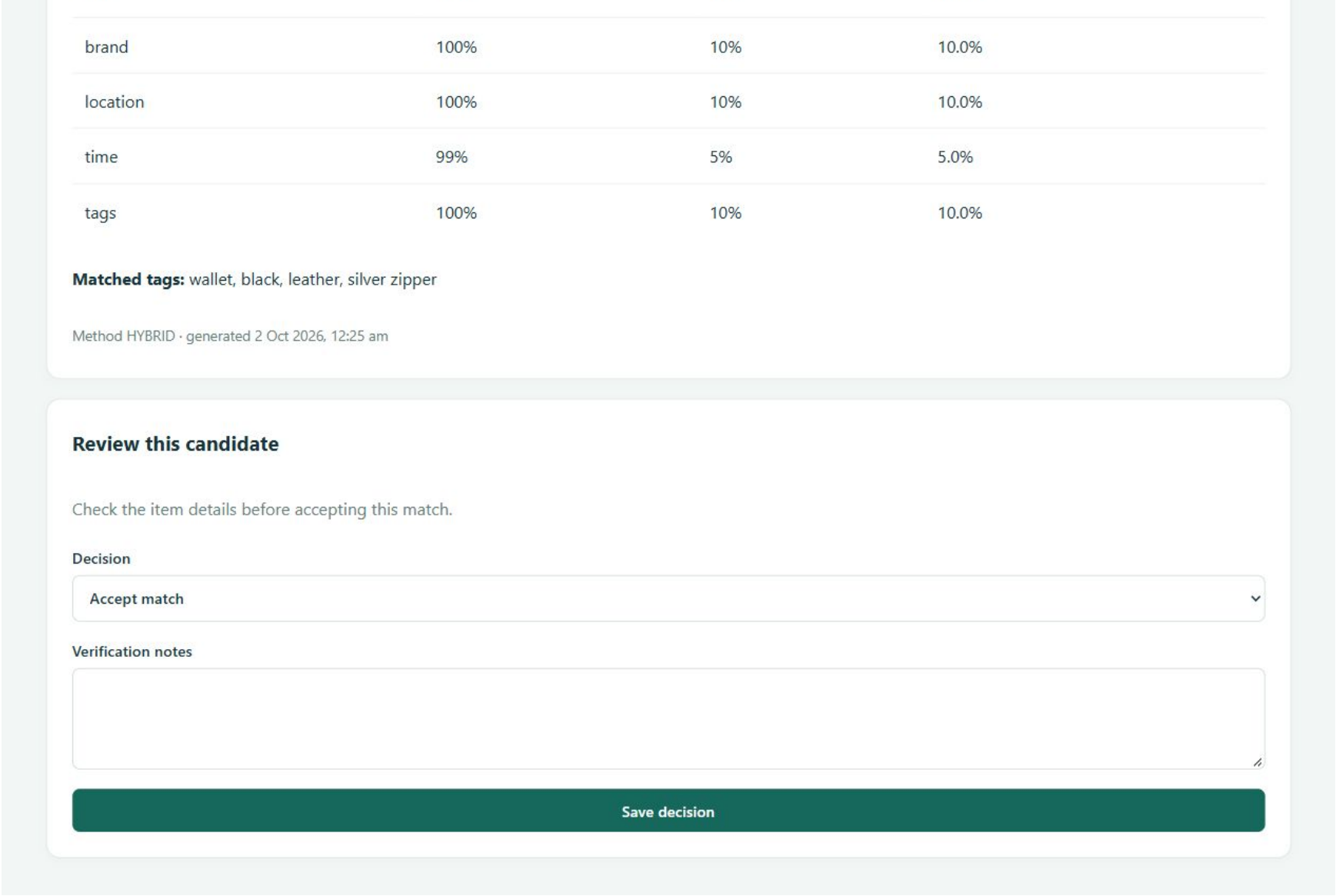
Appendix C Matching explanation and review | excerpt 1 of 2

Actual browser evidence: 04-match-evidence.jpg. Continuous excerpts; original file included in docs/screenshots.



Appendix C Matching explanation and review | excerpt 2 of 2

Actual browser evidence: 04-match-evidence.jpg. Continuous excerpts; original file included in docs/screenshots.



brand	100%	10%	10.0%
location	100%	10%	10.0%
time	99%	5%	5.0%
tags	100%	10%	10.0%

Matched tags: wallet, black, leather, silver zipper

Method HYBRID · generated 2 Oct 2026, 12:25 am

Review this candidate

Check the item details before accepting this match.

Decision

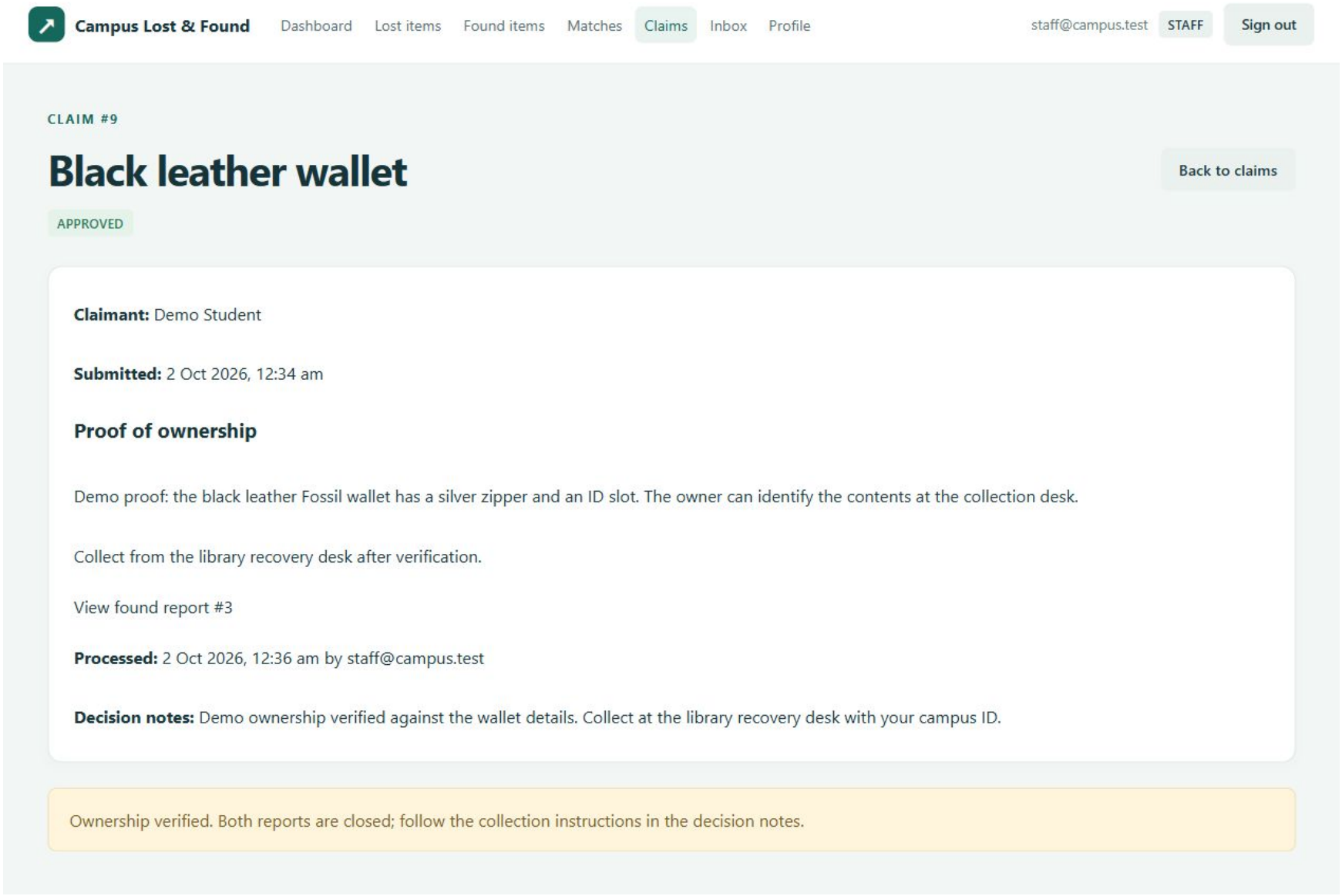
Accept match

Verification notes

Save decision

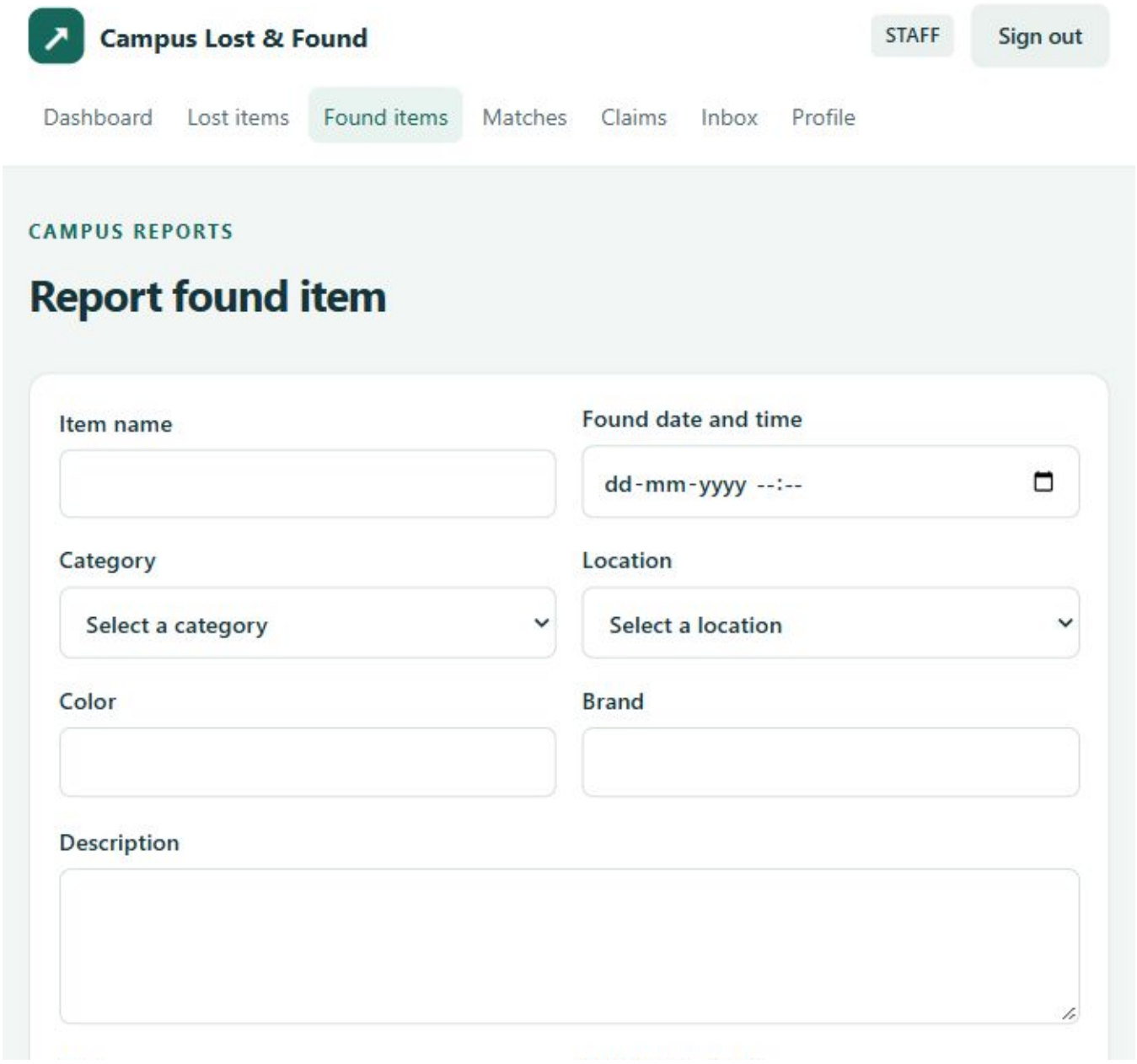
Appendix D Approved ownership claim

Actual browser evidence: 06-approved-claim.jpg. Continuous excerpts; original file included in docs/screenshots.



Appendix E Found report form | excerpt 1 of 2

Actual browser evidence: 14-found-report-form.jpg. Continuous excerpts; original file included in docs/screenshots.



Appendix E Found report form | excerpt 2 of 2

Actual browser evidence: 14-found-report-form.jpg. Continuous excerpts; original file included in docs/screenshots.

